

Assignment 3: Customer Support AI Assistant

Problem Statement: Create a POC for Customer support teams who often handle high volumes of repetitive inquiries, which can slow response times and impact service quality. Fully automated AI responses may lack accuracy, context, or appropriate tone for customer communication. This project aims to develop a **Customer Support AI Assistant** that drafts response suggestions while keeping humans in control. The system allows support agents to review, edit, and approve AI-generated replies before sending, ensuring efficiency, accuracy, and responsible use of AI in customer interactions.

Purpose:

This take-home assignment is designed to evaluate AI Full Stack Developer capability in building a production-oriented, end-to-end AI application. The focus is on frontend engineering excellence, API orchestration, LLM integration, application architecture, and software engineering.

You are expected to demonstrate how AI outputs are integrated into real user workflows with strong UX, reliability, and accuracy considerations.

Quick Facts:

Role level	AI Full Stack Developer
Submission format	Source code repository + written materials + reproducible instructions
Primary dataset	Pick any of the public datasets from Kaggle or Github or synthesized data.
LLM	Use any LLM

General Submission Rules

- Use only public, synthetic, or clearly de-identified datasets. Do not use protected health information (PHI) or any private/company-confidential data.
- Your solution must run from a clean environment using the commands you document in README.
- Pin or otherwise clearly control dependencies and random seeds where appropriate.
- You may use Python and any common open-source library. Do not require paid tools or paid cloud services.
- You may use AI-assisted coding tools if you wish, but you must be able to explain and defend every design choice, implementation detail, and debugging step during the live interview.
- Include a short design rationale, assumption log, and “what I would improve with more time” note in your submission.
- Include meaningful commit history that reflects how the solution evolved; avoid submitting a single final dump of code.

Assignment Objective

- This assignment evaluates frontend AI engineering capability, including prompt design, tool/function orchestration, backend integration (Node.js or FastAPI), and user-centric AI workflow implementation.
- Because the role operates in an enterprise environment, you must also include a lightweight but concrete responsible-AI and governance approach covering explainability, feedback handling, and accuracy.

Detailed Problem Statement

- **Frontend:** A React + TypeScript agent interface that displays customer queries, AI-drafted responses, and provides edit, approve, or reject actions with conversation history and status indicators.
- **Backend:** A lightweight Node.js or FastAPI service that manages conversation context, orchestrates LLM calls, enforces approval workflows, and exposes REST APIs for drafts, approvals, and feedback.

- **Data Sources:** Structured datasets containing essential customer support information—such as FAQs, policies, sample support tickets, AI-generated drafts, final agent responses, and feedback records—can be selected and stored locally. The system searches these datasets to retrieve relevant content, which is then used to generate accurate and context-aware response drafts for customer queries.
- **Human-in-the-Loop:** Mandatory agent review and approval of all AI-generated drafts, with full edit control, rejection options, and save data for future auditing.

What You Must Build

- The user interface should provide a text input box for entering customer queries along with a submit button. Upon submission, the query is validated against internal documentation, and a response draft is generated and displayed in a separate output section. The UI should also include Accept and Reject buttons, allowing agents to approve or decline the suggested response. Agent feedback and approval decisions are then stored in the backend data store for auditing and future improvement.
- A web-based interface for support agents to view customer queries (can be a text box which we can pass the query manually for the demo purpose), review AI-generated response drafts, edit content, and approve or reject responses before sending.
- An API layer to receive customer input, manage conversation context, invoke the AI model for drafting responses, and enforce human approval before final delivery.
- Locally stored or lightweight datasets containing FAQs, policies, sample tickets, and historical support content, which can be searched or retrieved to ground AI-generated drafts.
- A large language model configured to generate response suggestions only, using retrieved data and handling failures or low-confidence outputs safely.
- Mandatory agent review, full edit control, rejection options, and an audit trail tracking AI drafts versus final approved responses.
- Here once the response gets approved it gets saved to the backend (local file).
- Mechanisms to collect agent feedback, document limitations, and ensure responsible AI usage, accuracy, and transparency.

Required Deliverables

- Frontend and backend source code for the Customer Support AI Assistant.
- A short note describing prompt design, and any AI interaction improvements implemented.
- Source code repository
- README with environment setup instructions, application run commands, and configuration steps for the LLM and dataset.
 - Environment setup
 - How to run frontend and backend
 - How to configure API keys
- Architecture overview (diagram or text)
- Accuracy & limitations
- A brief discussion of risks such as incorrect answers, over-reliance on AI, or UX limitations relevant to the chosen dataset.

Constraints

- The application must be runnable on standard developer hardware using free, publicly available tools; no paid services or special infrastructure are required.
- Do not optimize only for UI polish or LLM output quality. Application correctness, reliability, UX clarity, and engineering maturity matter more than perfect AI responses.

- Use only public data, LLM models, and libraries.
- Any assumptions, limitations must be clearly documented.

Submission Checklist

- Application setup and run instructions documented in the README.
- Frontend and backend integration is clearly documented.
- LLM configuration, prompt flow, and dataset usage documented.
- At least one AI interaction improvement or control documented (accuracy improvement).
- Architecture and scale-out considerations documented (for example handling more users, larger datasets, or additional tools).